

Predicting the Popularity of Spotify Tracks

Data Science Project

Contents

Introduction	2
Data Overview & Cleaning.....	3
Data Exploration and Early Signals	4
Modelling Methods & Results.....	8
Discussion & Conclusions	11

Repository: [github/cathaldunne11/spotify_tracks](https://github.com/cathaldunne11/spotify_tracks)

Kaggle Data Source: [spotify-tracks-dataset](https://www.kaggle.com/datasets/cathaldunne11/spotify-tracks-dataset)

Contact Details

cathaldunne11@gmail.com

[My LinkedIn Profile](#)

[My Github Profile](#)

Introduction

The purpose of this project is to develop a model that is effective at predicting the popularity of songs in the [Spotify Tracks dataset](#) from Kaggle. Three different approaches are applied:

- Linear Regression using the Python library Scikit-Learn
- a Random Forests Regressor using Scikit-Learn
- Gradient Boosting using the Python library LightGBM

This report gives an overview and detail on all stages of the project, from data cleaning and exploratory data analysis to model development, results and conclusions.

This project was completed for the purpose of personal development and for continuing to improve my knowledge of problem solving with data.

Data Overview & Cleaning

In its raw form, the Spotify Tracks dataset contains 114,000 records and 20 fields (see [Spotify Tracks dataset](#) for a full list and description of the fields). For this project, the response variable is 'popularity' which takes a score 0-100 for how popular the track is.

Features include basic information such as the track and artist name but also common data points relevant to music such as song duration and key signature.

Additionally, there are a variety of numeric fields representing opaque yet relevant concepts such as 'danceability' and 'speechiness' of the track. The definitions of these fields, along with the response 'popularity' is not scrutinised in this project as the main focus is on practicing my data wrangling and modelling skills with Python.

Data Cleaning

Although there are 114,000 records in the dataset, there are only c. 89,000 distinct tracks. There is duplication on the genre, where if a track has multiple genres assigned to it then it will have multiple records. There are 114 distinct genres, so the genre field was one-hot-encoded to ensure distinctness for each track ID (a unique identifier for a combination of track name and album name).

At this stage it was noted that there are a handful of duplicate tracks in the dataset e.g. where the same track by the same artist appears on different albums and even has different values for popularity – assumed to be a function of the popularity of the album more-so than the track itself. For simplicity, these have been left as is.

Additionally, it was found that from the data in its original structure that there were exactly 1,000 records for each genre, which raises questions about representative a sample of music it is and how well any models could be generalised to a wider population when trained on this data.

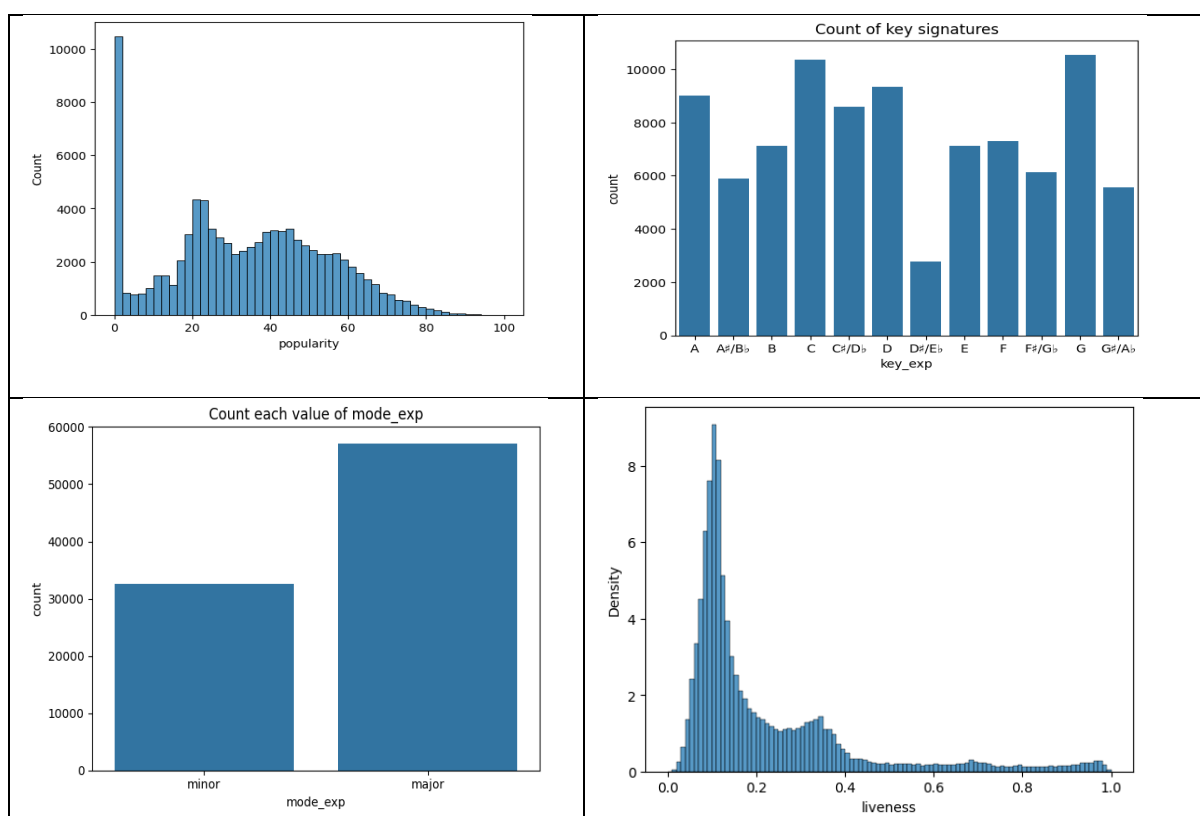
Data Exploration and Early Signals

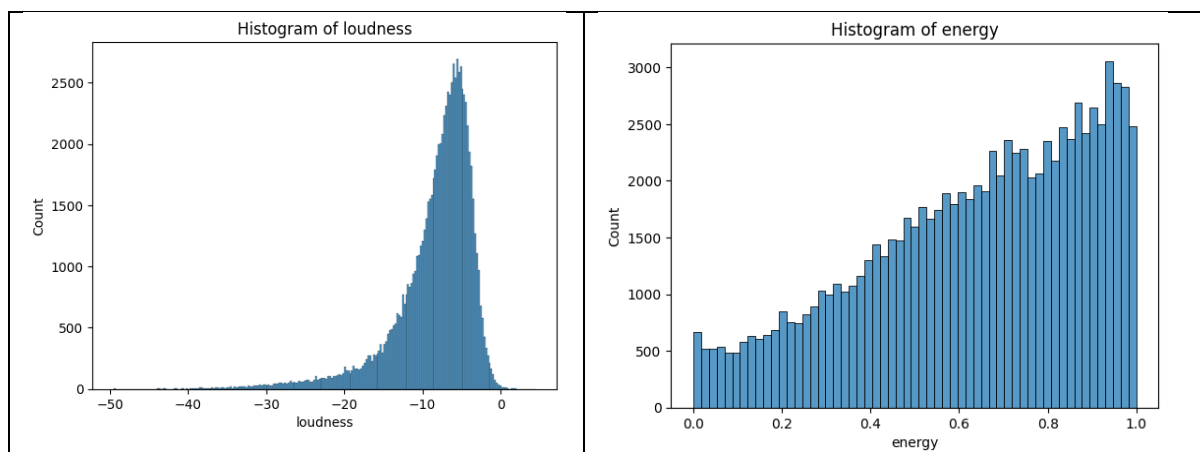
In this section, the initial findings from exploratory data analysis are presented. First, there is univariate analysis for selected variables. After this, there is bivariate analysis of the response variable against selected variables to explore the data deeper.

Univariate Analysis

Over 10% of tracks have a popularity of zero. Other than this, the data is bimodal around popularity of about 20 and 45. It is rare for tracks to have a popularity score of over 80.

Tracks are more likely to be major than minor, and are more likely to have a higher 'energy' score than lower.

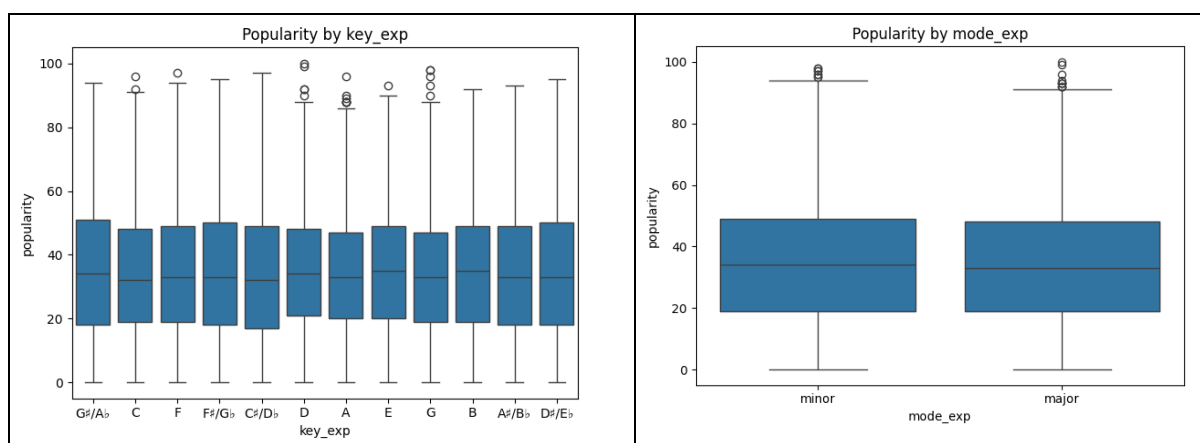


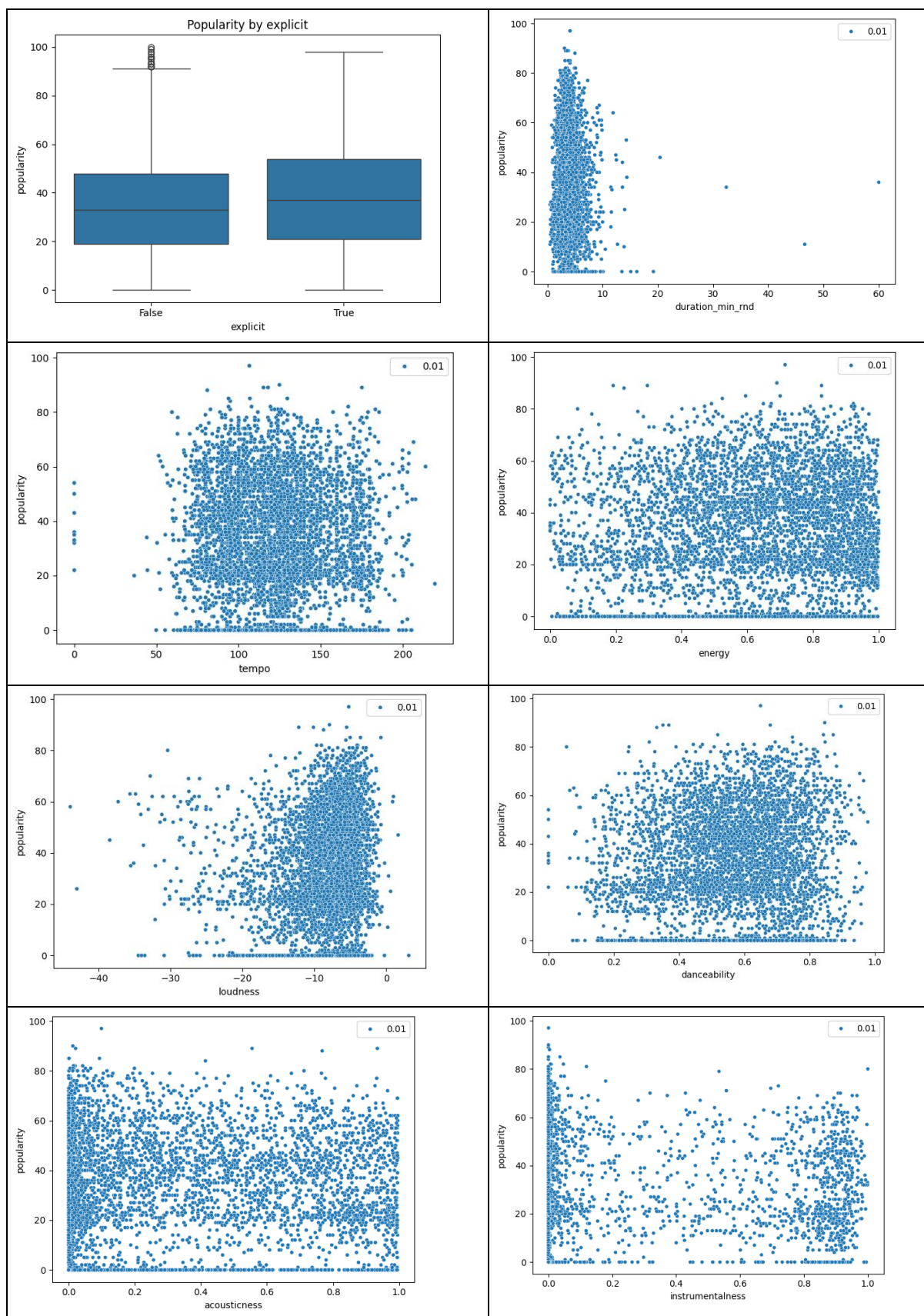


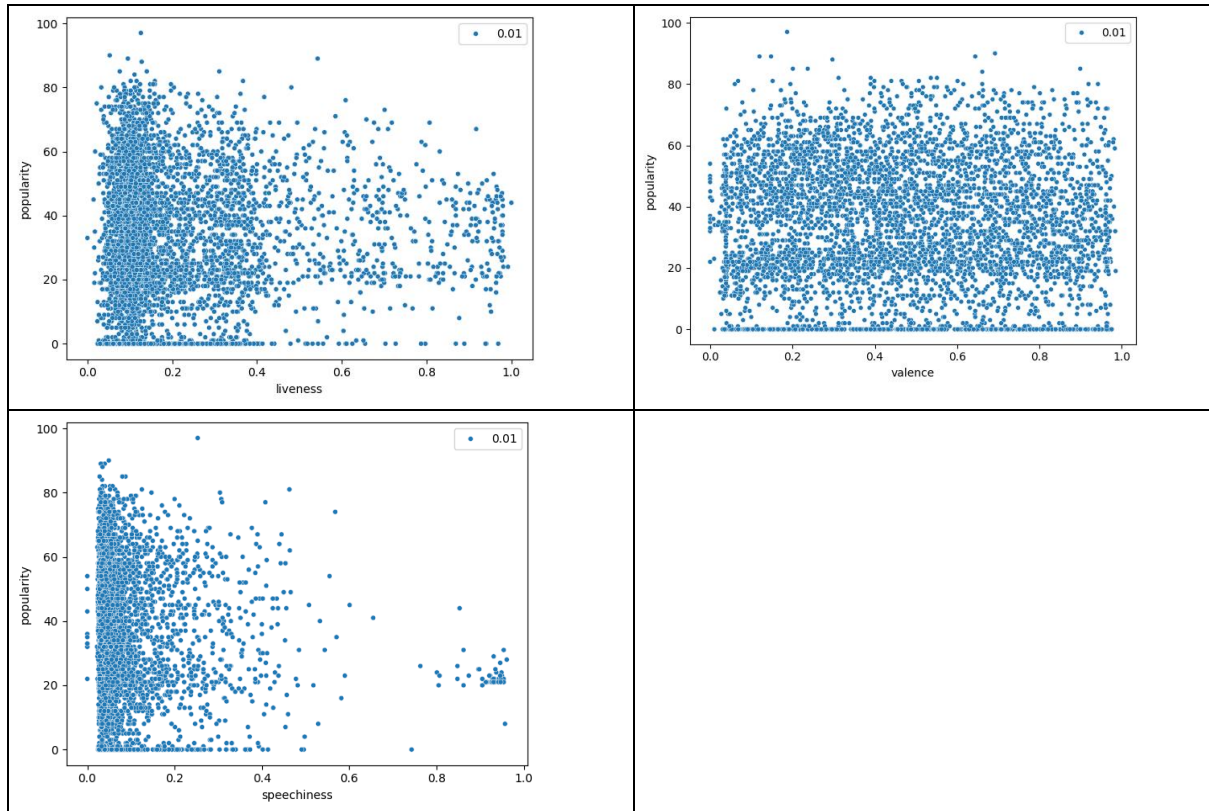
Bivariate Analysis

In this section, popularity is clashed against a variety of features to see what patterns become apparent. For scatterplots, the same random sample of 5,000 records has been used, to avoid the plots being too cluttered.

On initial inspection, there is little to no evidence of correlation between track popularity and the features. This suggests noisy data. Therefore, it may be difficult to predict popularity to a high degree of accuracy based on the features available.







Modelling Methods & Results

With popularity being a numeric variable, regression approaches are considered. A progressive modelling framework is implemented, with a simple parametric model implemented initially before progressing to more flexible non-parametric models. This approach is taken to adhere to the principle of parsimony – finding a simple model which performs well, and to optimise for a minimal total error while being mindful of the bias-variance trade-off.

First, a linear regression model is fitted to get a simple and fast baseline. After this, a random forest regressor and gradient boosting model are fitted to see if performance can be improved.

For model evaluation, both goodness-of-fit and prediction error metrics are considered. For consistency when evaluating models across methodologies, the Coefficient of Determination (R^2 score) and root mean squared error (RMSE) are used throughout. The R^2 score measures how much of the variance in the target variable that the model explains, with 1 being a perfect fit and 0 being no better than predicting the mean. RMSE measures the average magnitude of prediction errors, with lower being better. The objective is to maximise R^2 score while minimising RMSE.

Before fitting any models, an overview of the cleaned dataset is provided. The dataset has c. 89,000 records and 127 features. The features are:

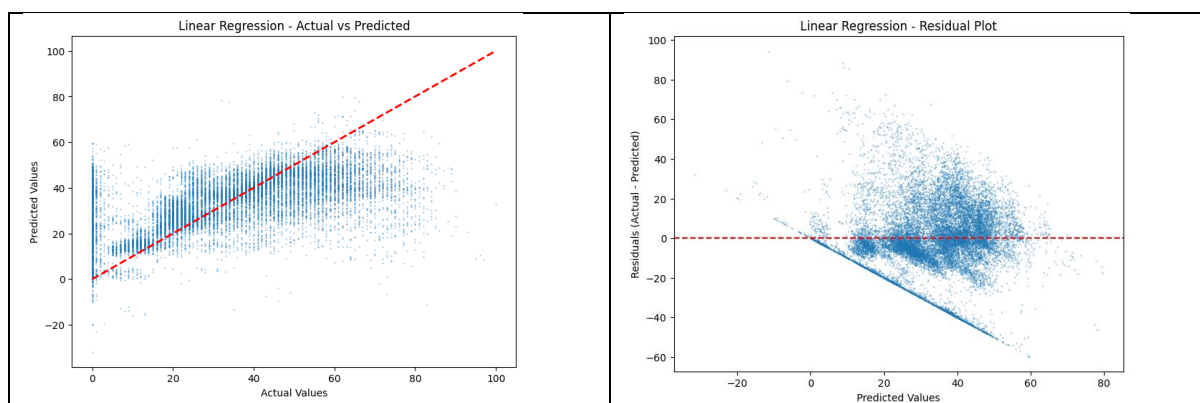
- 10 numeric ('danceability', 'speechiness' etc.)
- 114 binary features for genre – a track can have multiple genres
- 2 other binary features, one for modality and one for indicating if the track is explicit
- 1 multi-level categorical variable (key signature)

Linear Regression

To prepare the data, the key signature feature was one-hot encoded as all features need to be numeric. All other features were kept as they are. The data was then split into training (80%) and test (20%) datasets. The model was fit using scikit-learn's LinearRegression module and evaluated on the test data. It was found that the R^2 score was 0.35 (2 d.p.) and RMSE was 16.65 (2 d.p.).

From evidence in the below actuals vs. predictions plot and residual plot, the linearity and heteroscedasticity assumptions of linear regression are questionable. The slight n-shaped curve in the actual v predicted plot indicates non-linearity. On the residual plot, the residuals are more spread out above 0, indicating heteroscedasticity. Therefore, it is

justified to move to more flexible nonparametric models to capture nonlinearities and reduce prediction error.

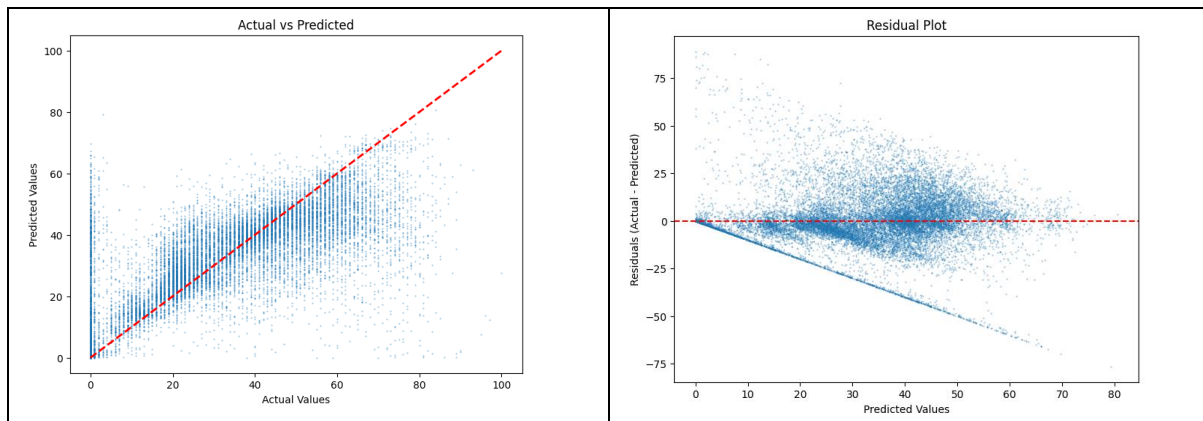


Random Forest Regressor

Now that the baseline model has been created, some more sophisticated models are applied. First, a Random Forest Regressor is fitted. Before implementing, the key signature variable was encoded as all values need to be numerical e.g. 'A' was mapped to '0', 'A#' to '1', 'B' to '2' etc. A simple baseline Random Forest with 100 trees was trained using scikit-learn's RandomForestRegressor module. An R^2 score of 0.50 was achieved with RMSE 14.51, an improvement in both metrics on the baseline linear regression model.

Next, Grid Search cross validation was applied to determine whether hyperparameter tuning could improve performance. For simplicity, just two hyper-parameters were chosen: number of trees and maximum tree depth. Training a model on all combinations of 50, 100, or 200 trees and a max tree depth of 'None' and 5, no additional gains were made in terms of R^2 score. The Grid Search results are shown below sorted by descending R^2 score.

	params	r2	mae	rmse
2	<code>{'max_depth': None, 'n_estimators': 200}</code>	0.500675	9.716577	14.531914
1	<code>{'max_depth': None, 'n_estimators': 100}</code>	0.498227	9.743513	14.567529
0	<code>{'max_depth': None, 'n_estimators': 50}</code>	0.493023	9.785062	14.642872
4	<code>{'max_depth': 5, 'n_estimators': 100}</code>	0.102238	15.947228	19.485864
3	<code>{'max_depth': 5, 'n_estimators': 50}</code>	0.102100	15.949155	19.487362
5	<code>{'max_depth': 5, 'n_estimators': 200}</code>	0.101983	15.947164	19.488636



Upon reviewing the residual plot and a plot of the actual vs. residual values for the baseline Random Forests model, it was decided going forward to remove tracks with 0 popularity from the dataset as there isn't strong evidence that they can be predicted well.

The same Grid Search was applied on the updated dataset, but only very marginal improvement was found, with an R^2 score only improving to 0.51 in the best case.

	params	r2	mae	rmse
2	<code>{'max_depth': None, 'n_estimators': 200}</code>	0.511356	8.844028	12.700461
1	<code>{'max_depth': None, 'n_estimators': 100}</code>	0.509228	8.864599	12.728115
0	<code>{'max_depth': None, 'n_estimators': 50}</code>	0.503917	8.906948	12.796823
4	<code>{'max_depth': 5, 'n_estimators': 100}</code>	0.155382	13.585629	16.697861
5	<code>{'max_depth': 5, 'n_estimators': 200}</code>	0.154969	13.589118	16.701937
3	<code>{'max_depth': 5, 'n_estimators': 50}</code>	0.154770	13.588961	16.703906

Gradient Boosting with LightGBM

As a final attempt to improve model performance, gradient boosting was applied to the dataset with zero popularity values excluded. LightGBM was the chosen algorithm/ library for its computational efficiency and performance against the baseline GradientBoosting Regressor in scikit-learn.

A baseline LGBM with below baseline metrics achieved an R^2 score of 0.53 and RMSE 12.7.

- Learning rate 0.05, maximum leaves 31, no maximum depth, feature and bagging fractions 0.8

Next, randomized search was applied to find improvements in hyper-parameter configurations. 30 configurations of the below hyper-parameters for the stated value ranges were fed into the randomized search.

- maximum number of leaves in a tree (range 8 to 128)
- fraction of features used by a tree (range 0.7 to 1)
- fraction of records used by a tree (bagging fraction, range 0.7 to 1)

However, the best performing iteration only achieved an R^2 score of 0.51, indicating that the out-of-the box LightGBM performed best and that additional hyper-parameter tuning in this instance is not worthwhile.

Discussion & Conclusions

For the data available, it has been found that it is challenging to develop a model that achieves very strong predictability for track popularity. However, the Gradient Boosting model that achieved 0.53 R^2 score is a moderate success for a couple of reasons. First, data describing music is likely to have significant amount of noise and randomness as its popularity is influenced by things like trends, culture, the time dimension of release and popularity. Additionally, concepts like 'danceability', 'speechiness', and even popularity itself are ill-defined here. Around 0.5 is likely an acceptable score as an R^2 score of closer to 1 may not be possible like it could be in physical sciences. Given that the upper limit of a variety of methods was around this 0.5 R^2 score mark, this may be close to the maximum achievable R^2 score for this problem.

For a future project where the same problem is addressed, the following changes of approach could be considered:

- Source a dataset with more track records so the models have more data with which to learn patterns, and that is more representative of music as a whole. Taking 1000 records from each of 114 genres is a flawed approach to building a representative dataset of music.
- Source data with better-defined and more tangible features.
- Consider ways to incorporate the album field as a categorical feature - from exploratory analysis this was a predictor of popularity when the same song was on different albums. For simplicity it was not considered this time as it is high-cardinality field.
- Improve the genre fields – tricky as this is far from an exact science. There was a lot of duplication or overlap (and hence multi-collinearity) between different genre fields once the dataset was one-hot encoded on genre (e.g. 'J-Pop' and 'J-Idol' were recorded as separate genres). Complete exploratory analysis on the duplication of these records and reasonably simplify these fields.
- Apply more time and resources to hyperparameter tuning with randomized search and grid search.